

**Student Name:**

**Weight:** %

**Student ID:**

**Marks:** /57

## **Assignment: Implement a simple hash function in Python**

### **Introduction**

Hashing is a technique used to map data of arbitrary size to fixed-size values. It is commonly used in data structures like hash tables for efficient searching and retrieval. In this activity, you will implement a simple hash function, observe its behavior, and explore the concept of salting for secure hashing.

### **Objective:**

- Understand the concept of hashing and hash functions
- Implement a simple hash function in Python
- Observe collisions and discuss how to handle them
- Understand the concept of salting and its importance in secure hashing

### **Materials Needed:**

- Computer with Python programming environment or Google COLAB
- Internet access for research and collaboration

### **Pre-Lab Preparation:**

1. Review the concept of hashing and its applications.
2. Familiarize yourself with hash functions and their properties.
3. Understand the problem of collisions in hash tables and common collision resolution techniques.
4. Read about salting and its role in secure hashing.

## **Instructions**

### **1. Implement a Simple Hash Function**

- Write a Python function that takes a key as input and returns a hash value between 0 and 9.
- Convert the key to a string and calculate the sum of the ASCII values of each character.
- Take the modulo of the sum with 10 to get a hash value between 0 and 9.

## 2. Hash a List of Keys

- Create a list of keys to hash, such as:
- `keys = ['apple', 'banana', 'orange', 'grape', 'kiwi', 'melon', 'pear', 'peach', 'mango', 'plum']`
- Hash each key using your hash function and print the key-hash pair.

## 3. Observe Collisions

1. Analyze the output and identify any collisions (multiple keys with the same hash value).
2. Discuss how collisions can be handled.

## 4. Implement Collision Handling

1. Modify your hash function to handle collisions using chaining (linked lists).
2. Create a Node class to represent each key-value pair and a HashTable class to handle insertion and retrieval.
3. Test your modified hash table by inserting the keys and their corresponding lengths as values.
4. Search for each key in the hash table and print the key-value pairs.

## 5. Generate a fixed-size output

1. Implement a hash function that incorporates salting to generate a fixed-size (64) output in a specific format.
2. The hash function should take a key and a salt as input and return a hash value as an 8-character hexadecimal string

## 6. Explore Salting

1. Research the concept of salting and its importance in secure hashing.
2. Discuss how salting helps protect against rainbow table attacks and makes it harder to crack hashed passwords.

## 7. Validate Mitigations

1. Modify your hash function to include salting.
2. Generate a random salt using `os.urandom()` and concatenate it with the key before hashing.
3. Test your salted hash function with the list of keys.

## 8. Document and Review

1. Create a well-documented Python script with your implementations of the hash function, collision handling, and salting.
2. Write a brief report explaining your observations, findings, and the importance of salting in secure hashing.
3. Submit your Python script and report for review.
4. Engage in discussions with your peers, providing feedback and sharing insights on their implementations and reports.

### Real-World Examples:

- 1) **Password Storage:** Hashing is commonly used to securely store user passwords in databases. Salting adds an extra layer of security to prevent attacks like rainbow table lookups.
- 2) **Data Integrity:** Hashing can be used to verify the integrity of data by comparing the hash values of the original and received data.
- 3) **Blockchain:** Hashing plays a crucial role in blockchain technology, where each block contains a hash of the previous block, ensuring the integrity and immutability of the chain.
- 4) **File Verification:** Hashing algorithms like SHA-256 are used to generate unique identifiers for files, allowing for efficient file comparison and verification.
- 5) **Caching and Indexing:** Hash tables are used in caching mechanisms and database indexing to enable fast data retrieval based on unique hash values.

## Marking Criteria

| Criteria                                   | Missing                                                                                      | Requires Improvement                                                                    | Good                                                                                     | Excellent                                                                                                                                                                                   | Marks |
|--------------------------------------------|----------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-------|
| <b>Simple Hash Function Implementation</b> | Poor (0-4): Hash function not implemented or fundamentally incorrect.                        | Fair (5-6): Hash function implemented but with significant errors or misunderstandings. | Good (7-8): Hash function implemented with minor errors or inefficiencies.               | Excellent (9-10): Correctly implements a hash function that takes a key and returns a value between 0 and 9. Function accurately converts key to string and calculates sum of ASCII values. | /10   |
| <b>Hashing a List of Keys</b>              | Poor (0-2): Failed to hash keys or output results correctly.                                 | Fair (3-4): Some keys hashed correctly, but with significant errors or omissions.       | Good (5-6): Most keys hashed correctly with minor errors in output.                      | Excellent (7-8): Correctly hashes all provided keys and prints key-hash pairs accurately.                                                                                                   | /8    |
| <b>Collision Observation and Analysis</b>  | Poor (0-2): Fails to identify collisions or provide meaningful analysis.                     | Fair (3-4): Identifies some collisions but lacks depth in analysis or discussion.       | Good (5-6): Identifies most collisions and provides a basic discussion on handling them. | Excellent (7-8): Thoroughly analyzes output, correctly identifies all collisions, and provides insightful discussion on handling collisions.                                                | /8    |
| <b>Collision Handling Implementation</b>   | Poor (0-4): Fails to implement collision handling or implementation is fundamentally flawed. | Fair (5-6): Basic implementation of collision handling, but with significant flaws.     | Good (7-8): Implements collision handling with minor errors or inefficiencies.           | Excellent (9-10): Successfully implements collision handling using chaining, with correct Node and HashTable classes. Insertion and retrieval work flawlessly.                              | /10   |

|                                                 |                                                                                           |                                                                                                  |                                                                                                 |                                                                                                                                                                                       |            |
|-------------------------------------------------|-------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|------------|
| <b>Fixed-size Output Hash Function</b>          | Poor (0-2): Fails to implement the function or implementation is fundamentally incorrect. | Fair (3-4): Basic implementation but with significant errors in salting or output.               | Good (5-6): Implements the function with minor errors in salting or output format.              | Excellent (7-8): Correctly implements a hash function that incorporates salting and generates a 64-bit (8-character) hexadecimal output.                                              | /8         |
| <b>Understanding and Application of Salting</b> | Poor (0-2): Poor understanding of salting or fails to implement it correctly.             | Fair (3-4): Basic understanding of salting but with significant gaps. Implementation has errors. | Good (5-6): Shows good understanding of salting with minor gaps. Implementation mostly correct. | Excellent (7-8): Demonstrates thorough understanding of salting, its importance in secure hashing, and protection against attacks. Correctly implements salting in the hash function. | /8         |
| <b>Documentation and Reporting</b>              | Poor (0-1): Poor or missing documentation and reporting.                                  | Fair (2): Basic documentation and reporting, lacking in detail or clarity.                       | Good (3-4): Good documentation and reporting with minor omissions or lack of depth.             | Excellent (5): Well-documented Python script with clear explanations. Comprehensive report on observations, findings, and importance of salting.                                      | /5         |
| <b>Total</b>                                    |                                                                                           |                                                                                                  |                                                                                                 |                                                                                                                                                                                       | <b>/57</b> |

Note: The instructor may adjust individual marks if required.